Summary of "Agentic State Management Lecture 10/14"

Wenquan Lu(wenquan), Savini Kashmira(savinik), Preetom Kumar Biswas(pkbiswas), Kennedy McCarthy(kenmcart)

Section 1: Research direction of the field

What is the research problem, and why is it important?

The shared research problem is how to manage reusable LLM inference state (KV caches) efficiently under limited GPU memory. Continuum addresses whether and how long to retain an agent's cache during tool calls, balancing faster continuation against memory availability for other jobs. Strata addresses efficient cache transfers and scheduling across SSDs, GPU and CPU memory for long-context requests. This is important because poor state management causes repeated computation, queueing delays, and data-transfer stalls, increasing latency and reducing throughput for agents and other context-intensive applications.

What was the state of related work *before* the lecture papers appeared?

Before Continuum and Strata, LLM serving research had progressed from efficient GPU memory management in vLLM/PagedAttention (2023) to reusable prefix caching in SGLang (2024), reducing memory waste and repeated computation. Systems such as Mooncake (2024) extended KV-cache capacity using distributed CPU memory and SSDs. Meanwhile, InferCept (2024) managed caches during external-tool interruptions, and Autellix (2025) introduced program-level scheduling to reduce agents' cumulative waiting time. However, gaps remained in coordinating cache retention with uncertain tool durations and repeated queueing, and in handling long-context cache transfers, and loading-aware batching.

What do the lecture papers solve, and where do they stand in the area?

Continuum solves the repeated recomputation and queueing delays agents experience when their KV caches are evicted during tool calls. It advances earlier agent-serving systems by jointly addressing cache retention and multi-turn completion time under uncertain tool durations. Strata solves the I/O bottlenecks of loading long-context KV caches from CPU memory and SSDs. It advances hierarchical caching from expanding storage capacity to efficiently using that capacity.

What follow-up work has built on the lecture papers?

HiSparse (August 2026) directly builds on Strata by adopting its GPU-assisted I/O technique to fetch KV entries from host memory during sparse-attention decoding (§3.4). It extends this mechanism with a bounded GPU cache, fused cache-miss handling, and layer-wise prefetching, allowing long-context decoding without keeping the full KV history on the GPU. MARS (2026) follows Continuum's direction of retaining KV caches across tool calls, broadening the problem to joint GPU–CPU scheduling. It considers runtime resource pressure when retaining or reclaiming cached state and coordinates GPU inference with CPU tool execution. It is a broader systems follow-up, rather than a direct extension of Continuum's TTL algorithm.

Section 2: Lecture papers

Paper 1: Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live

What is the problem of this paper?

Continuum investigates how to retain and schedule KV caches across tool-call pauses to minimize multi-turn agents' completion time. Evicting caches causes recomputation or reloading and repeated queueing, while retaining them occupies scarce GPU memory needed by other jobs. This tradeoff is difficult because tool durations vary unpredictably and delays accumulate across turns. Existing systems either evict caches at turn boundaries or retain them without adequately accounting for repeated queueing and uncertain tool durations. The paper provides convincing empirical motivation: Figure 3 shows substantial queueing delays even with CPU offloading, and Figure 4 demonstrates highly variable tool-call durations. These findings substantiate the problem for the evaluated workloads.

What is the solution of this paper?

Continuum introduces a **time-to-live (TTL) mechanism** that temporarily pins an agent's KV cache in GPU memory during tool calls. It chooses the retention duration by balancing the expected savings in recomputation/reloading and repeated queueing against the cost of occupying GPU memory, using observed tool-duration distributions. If the tool returns within the TTL, the next turn reuses the cache; otherwise, expiration makes the memory reclaimable. Combined with program-level first-come-first-served scheduling, this preserves continuity across turns while bounding memory retention when tool calls take unexpectedly long.

How is the solution evaluated?

Continuum is evaluated on SWE-Bench and BFCL workloads using Llama-3.1-8B/70B across A100, H100, and B200 GPUs. Comparisons against vLLM, Autellix, and InferCept measure job completion time and throughput, with additional live-agent tests against SGLang and Dynamo. Ablations and robustness tests examine TTL retention, scheduling, turn counts, and offloading. Results show up to 3.66× lower average completion time and 3.22× higher throughput. These tests support the scheduling benefits.

Overall, what are 1-2 shortcomings of this paper?

First, its sequential reason–tool–reason execution model limits generality: asynchronous agents and spawned subagents sharing a KV prefix introduce concurrent continuations with different state lifetimes, which its per-program TTL and FCFS scheduling do not explicitly address. Second, TTL decisions rely on historical tool-duration estimates and an approximate memory-contention model; changing execution conditions can lead to incorrect estimates.

Paper 2: Strata: Hierarchical Context Caching for Long Context Language Model Serving

What is the problem of this paper?

Long cached prefixes can make serving I/O-bound. Small GPU pages create fragmented transfers; larger pages improve transfers but reduce prefix-match precision and cache hit rate. Layer-wise overlap also fails when a request loads many cached tokens but computes few new ones, and concurrent requests may redundantly compute a shared prefix before its cache is ready, producing a "delay hit." In one LooGLE baseline configuration, transfers stall 74% of prefill execution time.

What is the solution of this paper?

Strata combines GPU-assisted I/O with cache-aware scheduling. A GPU kernel moves fragmented KV pages in parallel and transforms between the GPU's layer-first layout and a page-first host layout, preserving small GPU pages while improving bandwidth utilization. Its HiRadixTree tracks cache locations and prefixes awaiting computation, so the scheduler can defer duplicate prefix computation, balance cache loading against new-token computation, group requests sharing a cached context, and fill remaining loading stalls with decoding, which overlaps with CPU-GPU transfers.

How is the solution evaluated?

The SGLang implementation is evaluated primarily on H200 GPUs using Llama-8B, Qwen-14B, and Llama-70B; LooGLE, NarrativeQA, and ReviewMT test long contexts, and ShareGPT tests short contexts. Average time to first token (TTFT) and output-token throughput measure responsiveness and capacity. On Llama-70B/LooGLE, Strata achieves up to 5× the throughput of vLLM-LMCache and 3.75× that of TensorRT-LLM hierarchical caching at comparable TTFT, with broadly comparable short-context performance. Same-engine ablations strengthen attribution to both mechanisms, and optimized I/O alone on GH200 does not outperform full Strata on PCIe: hardware bandwidth cannot substitute for scheduling.

Overall, what are 1-2 shortcomings of this paper?

First, starvation prevention does not guarantee fairness or per-request latency objectives; average TTFT can hide badly delayed requests. Second, most experiments use synthetic Poisson arrivals and generous host caches (1 TB of pinned DRAM on H200), leaving performance under bursts, changing reuse patterns, and tighter memory budgets less established.

Section 3: Lecture discussions

1. For Continuum Section 3.1, the cost is calculated using the full TTL, while in per-turn eviction the agent's next request scheduled is often earlier than the TTL. Does that overestimate the cost?

The true cost depends on many other factors, and overestimating it is the safer choice. The cost model is an approximation; future work could account for the time spent switching between tools.

2. For Strata, should the eviction policy consider context size, such as a 100K-token versus a 2K-token context?
Recency alone may not be enough, since restoring a large context costs much more than a small one and could worsen delay hits.
3. The authors ran their agents with GPT-5. Does the tool timing in their traces match how a Llama agent would behave?
Continuum does not change model inference or fix model nondeterminism. It only optimizes how incoming requests are scheduled, regardless of which model is used.
4. Tool runtime also depends on the command and its arguments. Can Continuum account for that?
Some weighting by command could help, and future work could make the predictions argument-aware.
5. Continuum uses first-come, first-served (FCFS) scheduling, while Strata defers requests with large contexts. Which approach is better?
Continuum avoids starvation and lets tool calls resume quickly, while Strata may defer a request to a later stage. The better choice depends on the specific workflow.
6. For Strata, does the heavy host-to-GPU transfer workload create a bottleneck?
It could become a bottleneck. Two CUDA blocks for loading and one for backup was the best configuration without taking up too many GPU resources.
7. Continuum seems simple and sequential. Does it still work when many subagents are spawned?
Continuum works for simple workflows. Workflows with dynamic depth and width form dynamic DAGs, which are very hard to handle. Professor Chowdhury expands this into a full-class discussion on managing KV cache for complex workflows. Some suggest partitioning of KV cache, while others suggest training a small policy/LLM-as-judge to predict time. The professor admits it is a hard problem.
8. Continuum depends on history to predict TTLs. What happens without history, or with a large error bar?
Without history, it falls back to the baseline. An LLM could estimate tool time through in-context learning, but the system must recover safely from wrong predictions.

Section 4: Proposal for follow-up work

Strata improves average TTFT but does not guarantee per-request latency; in lecture, Continuum's FCFS scheduling avoided starvation, while Strata's deferral could delay individual requests. Extending Section 1's focus on cache transfers and scheduling, we ask whether cache-efficient batching can meet individual TTFT targets when short requests compete with long cached contexts under bursty traffic. Our contribution is deciding when deferral is safe, not deadline scheduling itself.
We would extend HiRadixTree with cache-ready times and request deadlines, prototyped on one Llama-70B deployment serving mixed LooGLE and ShareGPT traffic. A shared-prefix request is deferred only while its estimated first-token time still meets its target. Urgent requests receive priority for computation and a reserved share of transfer bandwidth; Strata's efficiency-oriented grouping uses the rest.
We would first confirm that bursty mixes push short requests past their TTFT targets under unmodified Strata. With identical engine, kernels, and memory budgets, we would compare Strata, FCFS, a fixed deferral cap, and our policy across arrival rates, burstiness, prefix popularity, and host-cache sizes, measuring TTFT-target attainment, P99 TTFT by context length, throughput, and scheduler overhead. Gains beyond the fixed cap would support the approach; similar results, or gains lost to prediction errors or reduced throughput, would challenge it.

Section 5: A reflection on AI usage

We find that AI can generally retrieve accurate information from the paper and produce responses with fewer hallucinations due to the integration of tool-use capabilities. This is very relevant to the two papers discussed above. We found that AI spent substantial time searching through websites and literature while assisting with writing Section 1, and also spent time querying the uploaded PDFs while assisting with summarization in Section 2. This tool-use time further reinforces the importance of "agentic state management." However, AI responses can sometimes be vague and require explicit prompting. As an example, GPT initially described Continuum's solution in broad terms without mentioning its time-to-live mechanism, requiring manual correction.